Lecture 4: Assembly

Professor G. Sandoval

Some slides adapted by G. Sandoval for CS3224, from slide by Brendan Dolan-Gavitt and Bryant and O'Hallaron, Computer Systems: A programmer's Perspective, Third Edition

How do I feel about assembly?

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

OS Abstractions of HW

Hardware Resource	OS abstraction
CPU	Processes / Threads
Main memory	Address spaces
Disk	File system
Network	Sockets / ports
Devices	Virtual devices

HW Support for OS Abstractions

OS Service	Hardware Support
Isolation between processes	Kernel/user mode Protected instructions Memory management unit
Virtual memory	MMU / TLB
System calls	Traps
Asynchronous I/O	Interrupts / DMA
CPU scheduling / accounting	Timer interrupts
Synchronization	Atomic instructions

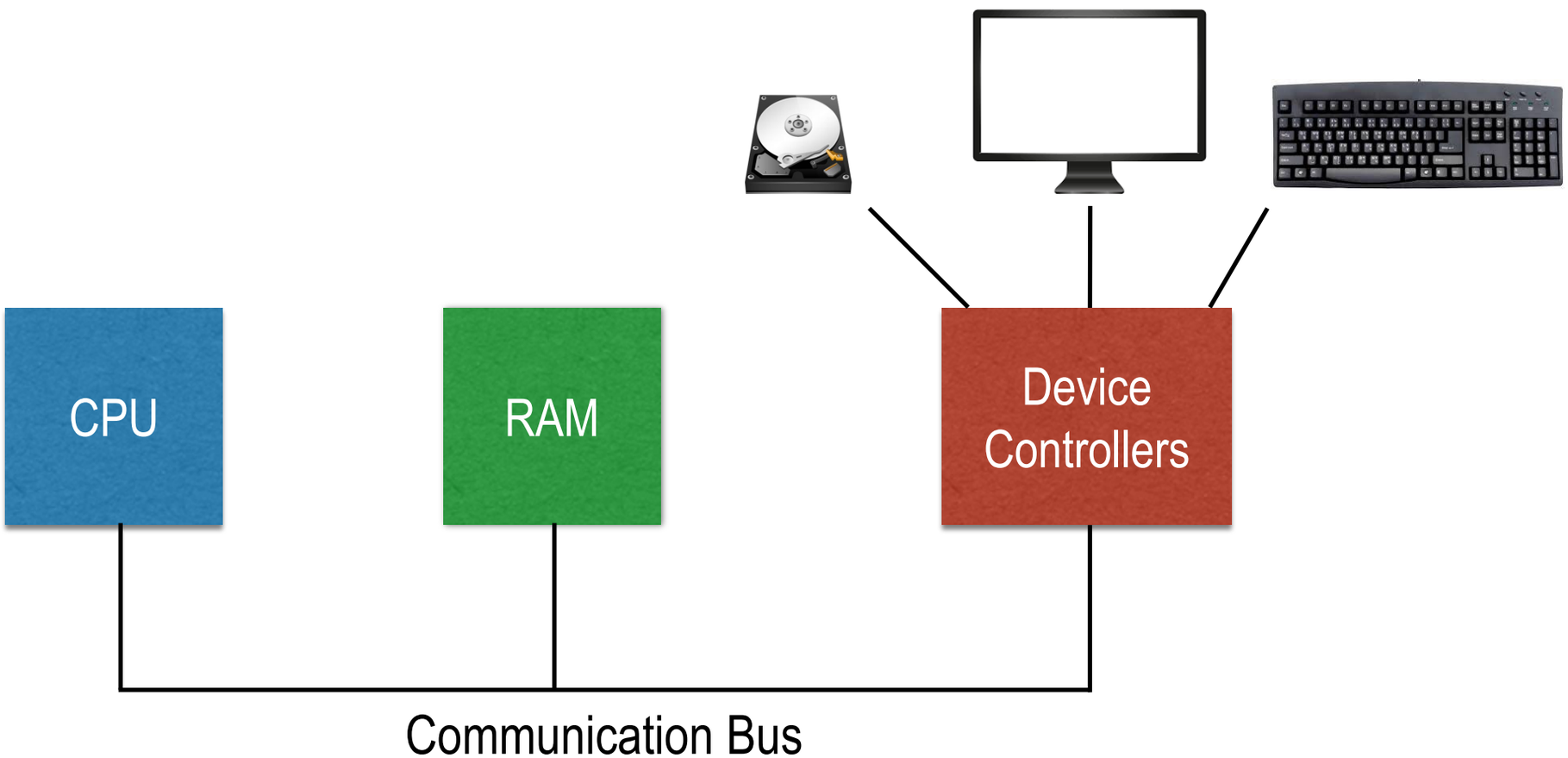
Today: Machine Programming Basics

- **History of Intel processors and architectures**
- **C, assembly, machine code**
- **Assembly Basics: Registers, operands, move**

The Original IBM PC : 1984

PC Architecture

(vastly simplified)



PC Architecture

(slightly less simplified)

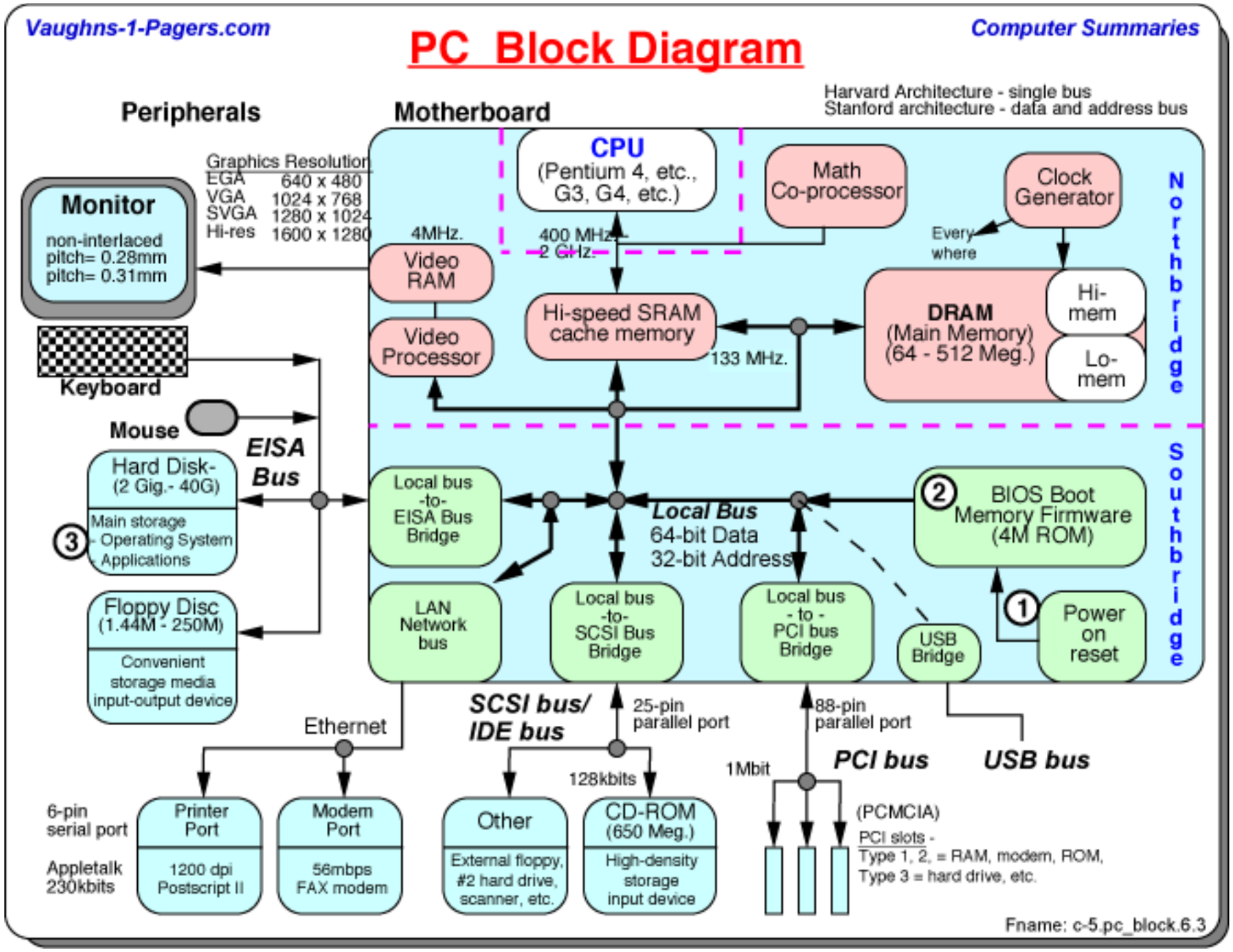


Diagram by Vaughn Aubuchon

Intel x86 Evolution: Milestones

Name	*Date*	*Transistors*	*MHz*
8086	**1978**	**29K**	**5-10**

- First 16-bit Intel processor. Basis for IBM PC & DOS
- 1MB address space

386	**1985**	**275K**	**16-33**

- First 32 bit Intel processor , referred to as IA32
- Added “flat addressing”, capable of running Unix

Pentium 4E	**2004**	**125M**	**2800-3800**

- First 64-bit Intel x86 processor, referred to as x86-64

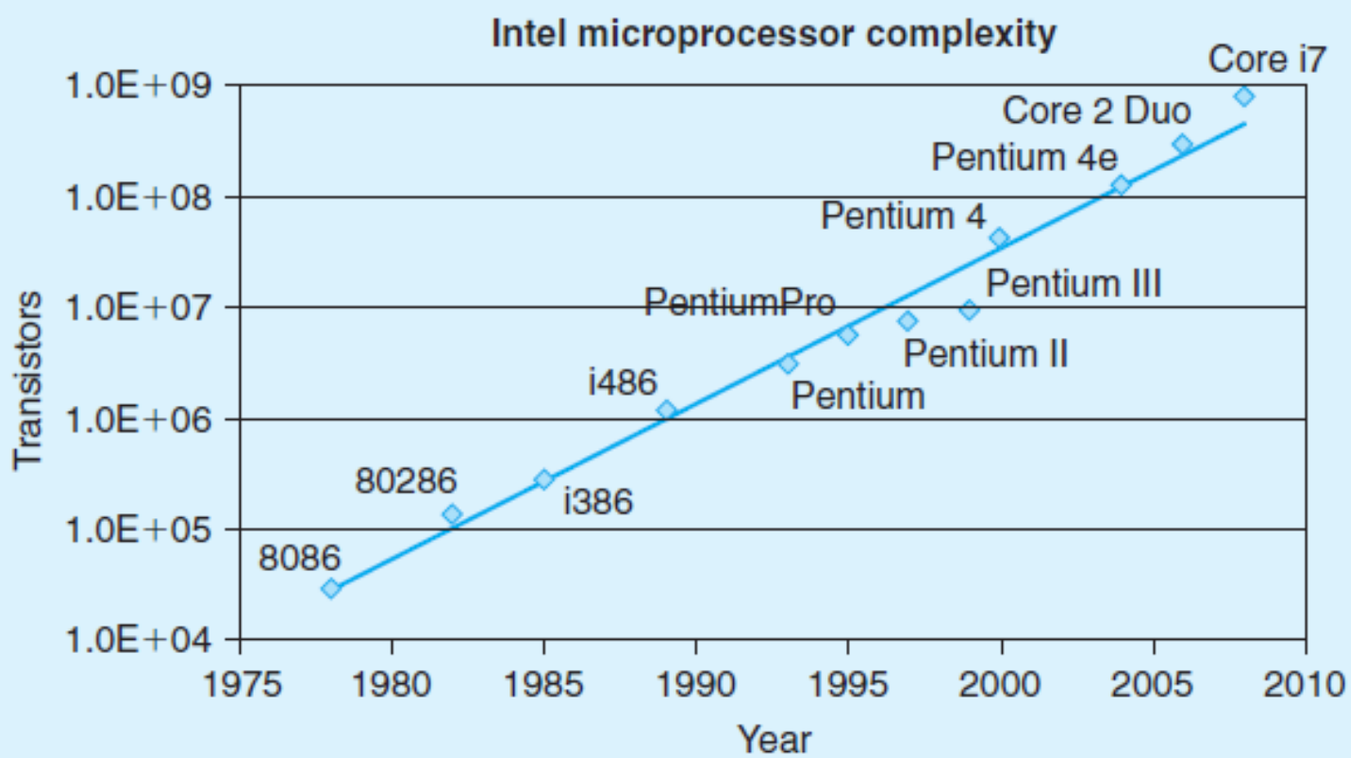
Intel x86 Evolution: Milestones (Cont)

Name	*Date*	*Transistors*	*MHz*
Core 2	**2006**	**291M**	**1060-3500**
Core i7	**2008**	**731M**	**1700-4400**
Core i9	**2017**	**3.5 – 7 Billion**	**2900-3300**

- First multi-core Intel processor
- Four cores
- 10 – 18 cores. First 18 core processor

Moore's Law

Aside Moore's law



If we plot the number of transistors in the different Intel processors versus the year of introduction, and use a logarithmic scale for the y-axis, we can see that the growth has been phenomenal. Fitting a line through the data, we see that the number of transistors increases at an annual rate of approximately 38%, meaning that the number of transistors doubles about every 26 months. This growth has been sustained over the multiple-decade history of x86 microprocessors.

Source: Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective. Third Edition

The BIOS

- **Responsible for the very first bit of code run on the system**
- **Does some simple initialization of basic hardware**
- **Also provides an API for hardware access**
  - But this is typically not used except for the very early stages of boot now
- **Knows how to find media to boot off of, load code from them into memory, and hand off control**

What Does a CPU Do?

- **Runs a simple algorithm:**

```
pc = 0
while True:
   fetch instruction at address pc in RAM
   decode instruction
   execute instruction
   if instruction did not modify pc:
       pc = pc + sizeof(instruction)
```

CPU

Nanosecond = billionth of a second = 10^-9

- **Contains a (tiny) bit of local storage (registers)**
  - ~1-3 nanoseconds to access
- **Contains a little more on-chip storage (cache)**
  - ~2-8 ns to access L1, 5-12 ns to access L2
- **For comparison:**
  - RAM is 10-60 ns,
  - SSD is 100 µs, (microseconds)
  - disk is 3-20 **ms**

microsec = 1 millionth of a second = 10^-6

millisecond = 1 thousand of a second = 10^-3

- **ASIDE:**
  - **Grace Hopper: Nanosecond vs Millisecond**.

Talking to Devices

- **Memory-mapped I/O:** A portion of the address space is mapped to a device, rather than to main memory
- **Port I/O:**
  - Special CPU instructions to talk to a fixed number of I/O ports
  - Only 1024 I/O ports defined
- **Interrupts: how devices talk back**
  - When a device wants to get the attention of the CPU, it can raise an *interrupt* (IRQ)

RAM Layout (32-bit)

Start	End	Description
0x00000000	0x0009FFFF	Low Memory
0x000A0000	0x000BFFFF	VGA Memory
0x000C0000	0x000EFFFF	16-bit Devices
0x000F0000	0x000FFFFF	BIOS ROM
0x00100000	?	Extended Memory
...		
0xC0000000	0xFFDFFFFF	PCI Devices
0xFFE00000	0xFFFFFFFF	High BIOS

On Linux, you can look at /proc/iomem for a detailed list

Today: Machine Programming I: Basics

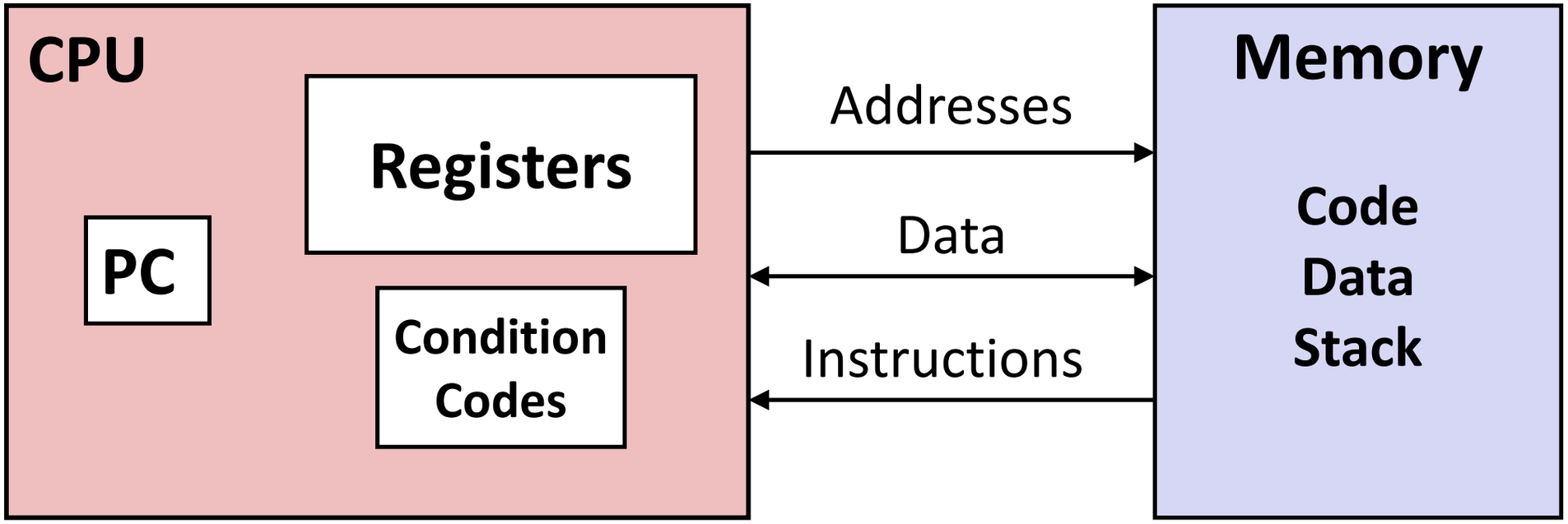
- History of Intel processors and architectures
- **C, assembly, machine code**
- Assembly Basics: Registers, operands, move
- Arithmetic & logical operations

Definitions

- **Code Forms:**
  - Machine Code: The byte-level programs that a processor executes
  - Assembly Code: A text representation of machine code

- **Example Intruction Set Architecture (ISA)s:**
  - Intel: x86, IA32, Itanium, x86-64
  - ARM: Used in almost all mobile phones

Assembly/Machine Code View

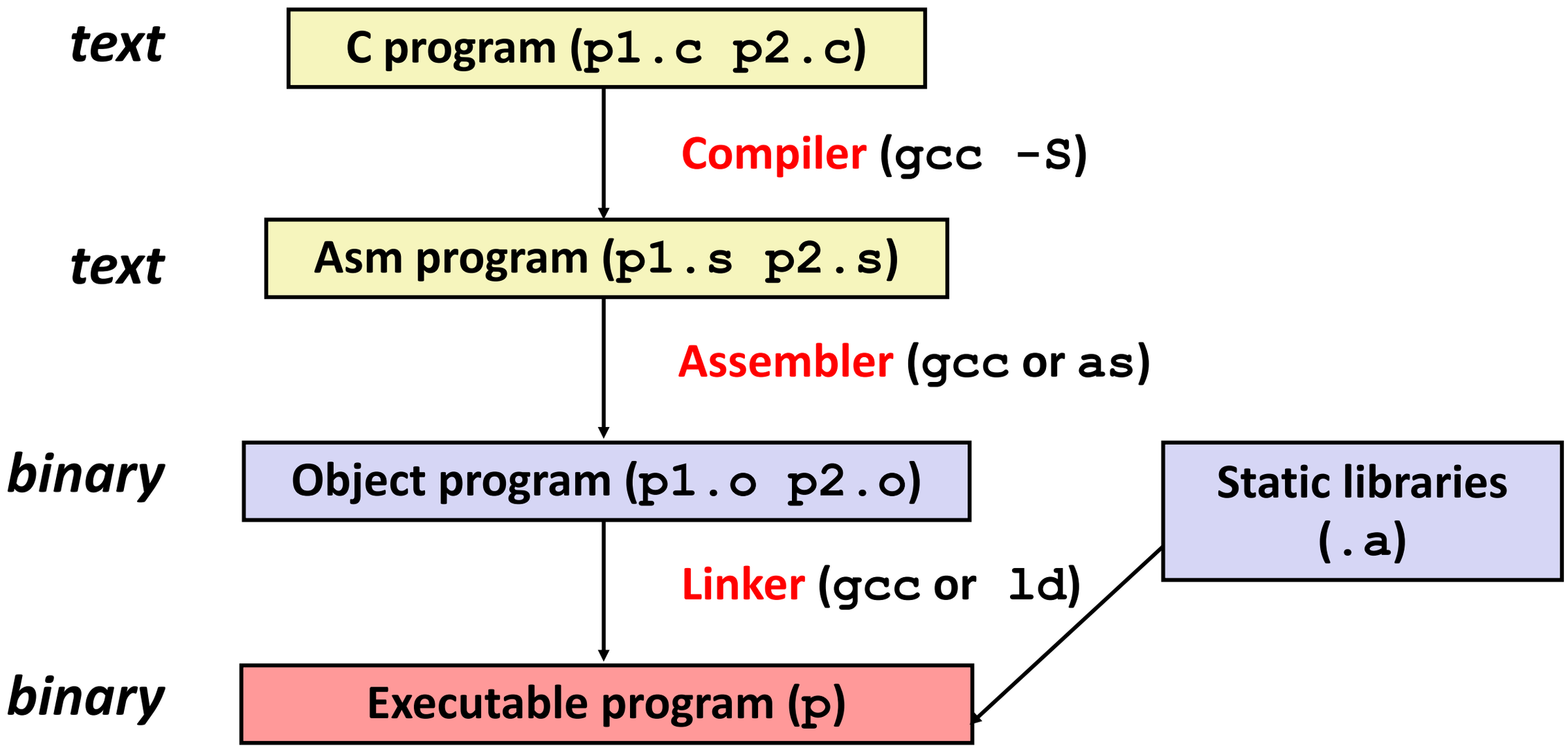


Programmer-Visible State

- **PC: Program counter**
  - Address of next instruction
- **Registers**
  - Heavily used program data
- **Condition codes**
  - Store status information about most recent arithmetic or logical operation
  - Used for conditional branching
- **Memory**
  - Byte addressable array
  - Code and user data
  - Stack to support procedures

Turning C into Object Code

- Code in files `p1.c p2.c`
- Compile with command: `gcc p1.c p2.c -o p`
  - Put resulting binary in file `p`



Machine Instruction Example

```
*dest = t;
```

me

```
movq %rax, (%rbx)
```

me

```
0x40059e:   48 89 03
```

comp

- **C Code**
  - Store value **`t`** where designated by **`dest`**
- **Assembly**
  - Move 8-byte value to memory
    - Quad words in x86-64 parlance
  - Operands:

    `t`: Register **`%rax`**

    `dest`: Register **`%rbx`**

    `*dest`: Memory **`M[%rbx]`**
- **Object Code**
  - 3-byte instruction
  - Stored at address **`0x40059e`**

Disassembling Object Code

Disassembled

```
0000000000400595 <sumstore>:
  400595:  53                  push   %rbx
  400596:  48 89 d3            mov    %rdx,%rbx
  400599:  e8 f2 ff ff ff      callq  400590 <plus>
  40059e:  48 89 03            mov    %rax,(%rbx)
  4005a1:  5b                  pop    %rbx
  4005a2:  c3                  retq
```

- **Disassembler**

  `objdump -d sum`

  - Useful tool for examining object code
  - Analyzes bit pattern of series of instructions
  - Produces approximate rendition of assembly code
  - Can be run on either `a.out` (complete executable) or `.o` file

Alternate Disassembly

Object

```
0x0400595:
    0x53
    0x48
    0x89
    0xd3
    0xe8
    0xf2
    0xff
    0xff
    0xff
    0x48
    0x89
    0x03
    0x5b
    0xc3
```

Disassembled

```
Dump of assembler code for function sumstore:
 0x0000000000400595 <+0>: push   %rbx
 0x0000000000400596 <+1>: mov    %rdx,%rbx
 0x0000000000400599 <+4>: callq  0x400590 <plus>
 0x000000000040059e <+9>: mov    %rax,(%rbx)
 0x00000000004005a1 <+12>:pop    %rbx
 0x00000000004005a2 <+13>:retq
```

- **Within gdb Debugger**

  `gdb sum`

  `disassemble sumstore`

  - Disassemble procedure

  `x/14xb sumstore`

  - Examine the 14 bytes starting at `sumstore`

Today: Machine Programming I: Basics

- History of Intel processors and architectures
- C, assembly, machine code
- **Assembly Basics: Registers, operands, move**

IA-32 (X86) ARCHITECTURE

IA-32 ARCHITECTURE

- **Evolutionary Design**
  - Starting in 1978 with 8086
  - Added more and and more feature as time goes on
  - Sill support old obsolete features
- **Complex Instruction Set Computer(CISC)**
  - Many different instructions with different formats
  - Only small subset found in Linux programs
  - Hard to match RISC perf but Intel has done it !
- **Lots of architecture improvements**
  - Pipelining, branch prediction, hyperthreading and multicore
- **From programmer's POV it has not changed substantially**
  - Except for intro of high perf instructions.

Modes of Operation

- **Protected Mode**
  - Native mode (windows, Linux), full features separate memory
- **Virtual 8086 mode**
  - Hybrid or protected
  - Each program has it’s own 8086 computer
- **Real Address mode**
  - Native MS-DOS
- **System management mode**
  - Power management, system security, diagnostics

x86 (32-bit)

- General purpose registers:
  EAX, EBX, ECX, EDX, ESI, EDI
  (though some have historical uses: ECX is used for counters, ESI & EDI are source and destination for string operations)

- Special-purpose registers:
  EBP (base pointer), ESP (stack pointer)

- Flags register: EFLAGS

- Segment registers: CS, SS, DS, ES, FS, GS

- Instruction pointer/program counter: EIP

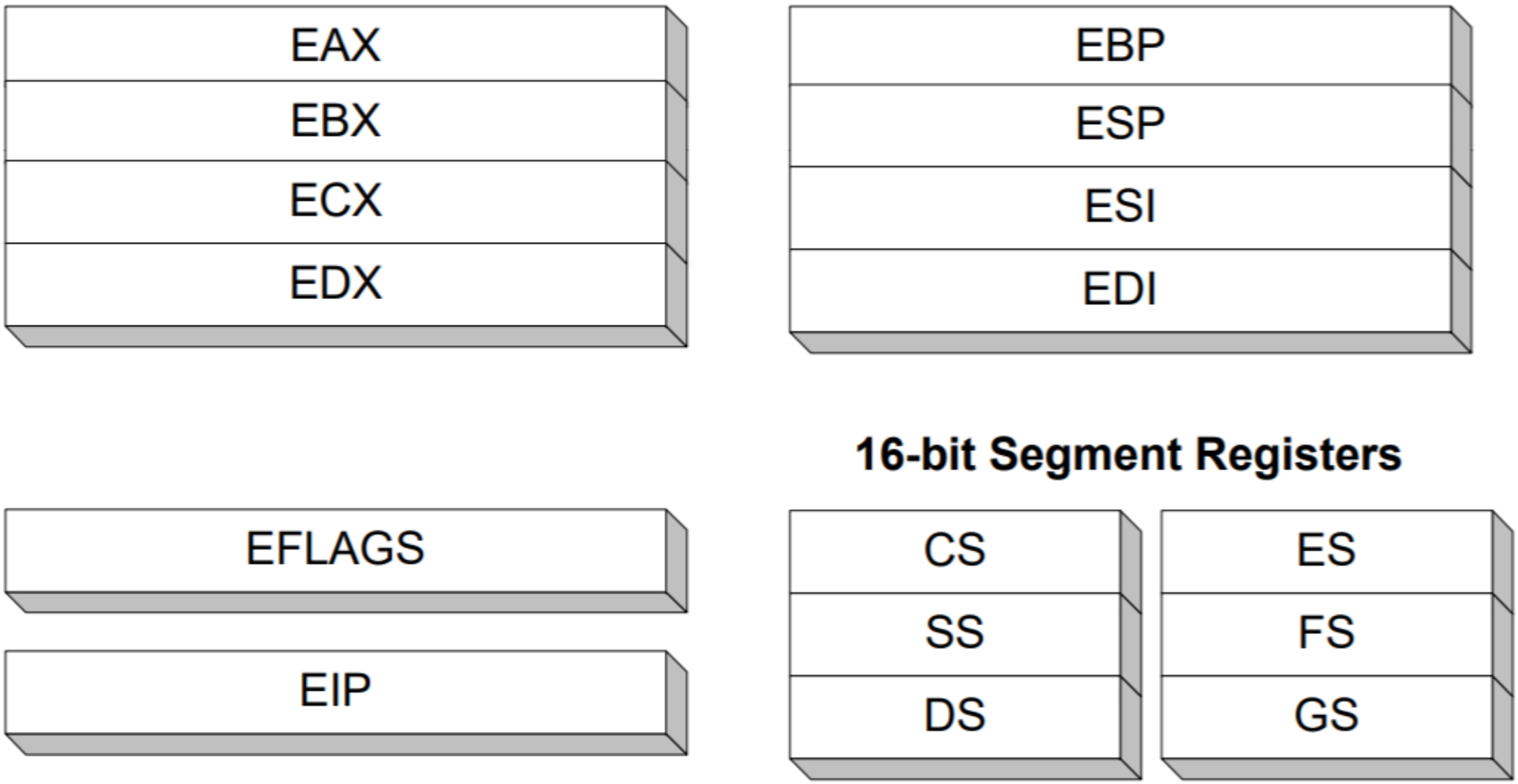
- Control registers: CR0-CR4

Addressable Memory

- **Protected Mode**
  - 4 GB Address space
  - 32 bit addresses

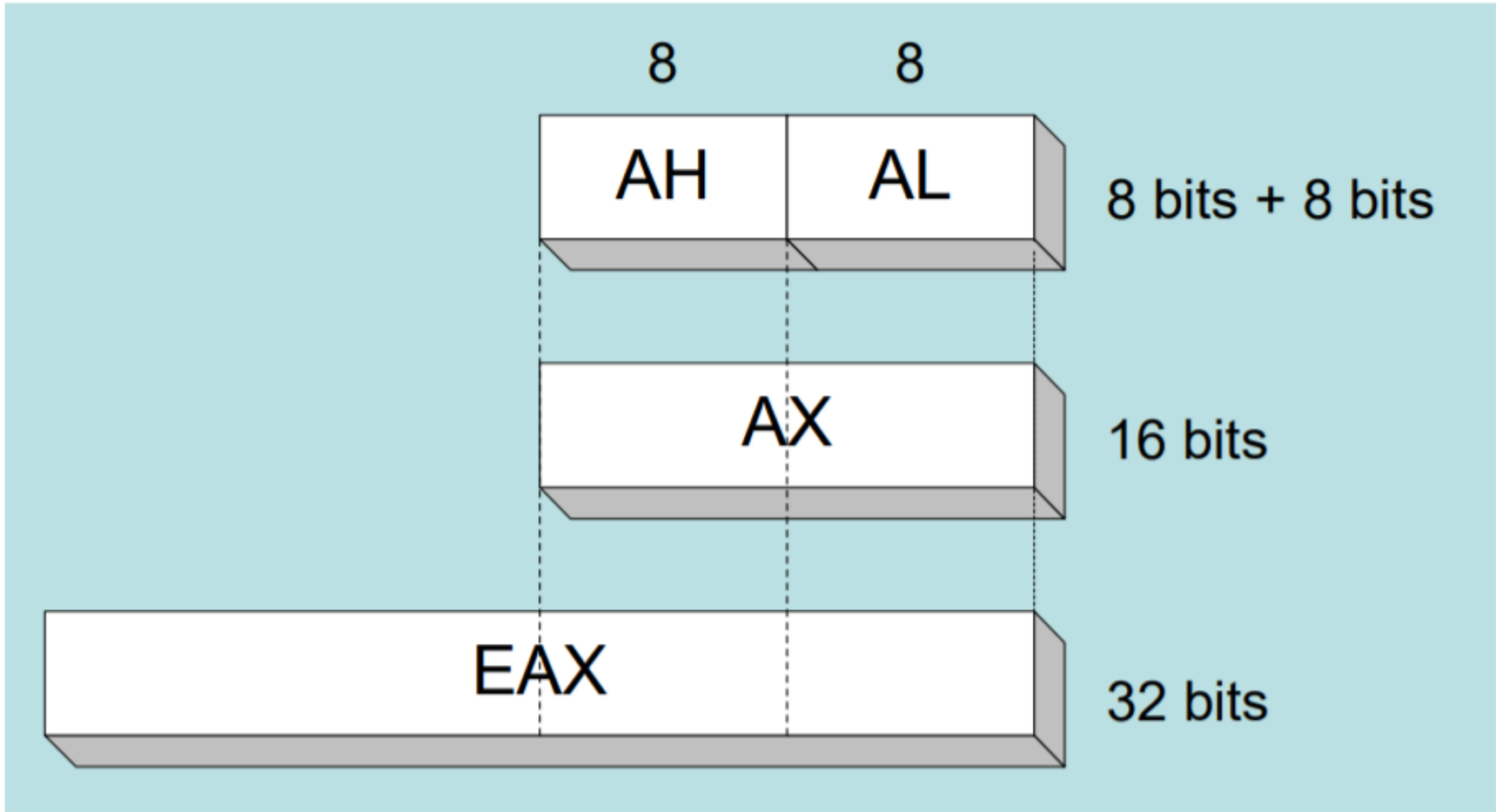
- **Real address and Virtual 8086 Modes**
  - 1 MB address space
  - 20 bit addresses

32 bit General Purpose Registers



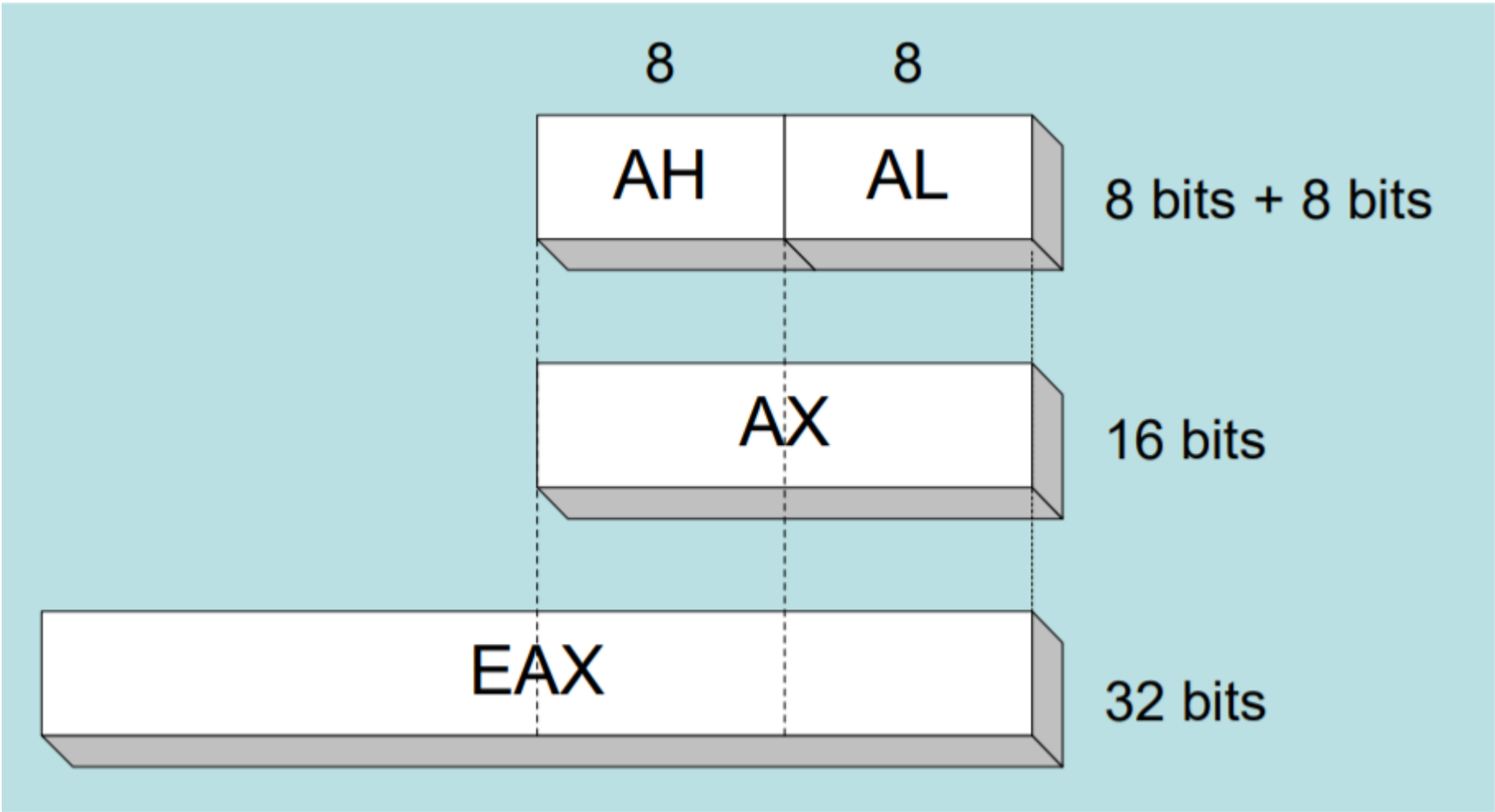
Accessing parts of registers

- Use 8 bit name, 16-bit name or 32-bit name
- Applies to EAX, EBX, ECX and EDX



Accessing parts of registers (2)

32-bit	16-bit	8-bit (high)	8 bit (low)
EAX	AX	AH	AL
EBX	BX	BH	BL
ECX	CX	CH	CL
EDX	DX	DH	DL



Index and base registers

- **Some registers only have a 16-bit name for their lower half (not 8 bit aliases). The 16-bit registers used only in real-address mode.**

32-bit	16-bit
ESI	SI
EDI	DI
EBP	BP
ESP	SP

X86 INSTRUCTIONS (REFRESHER)

x86 Instructions

- **Instructions are *variable length***
- **Conventions:**
  - regN means a register of N bits
  - mem means memory
  - constN means a constant of N bits

Note: x86 is *BIG*

- When they stopped printing paper manuals (c. 2010), occupied Intel manual occupied 5 volumes
- Two volumes (1300 pages as of June 2015) dedicated just to describing instructions
- We will only cover a minimal subset of x86 here; if you have questions about a particular instruction the Intel manuals are actually pretty good

Intel® 64 and IA-32 Architectures Software Developer's Manual
VOLUME 3B: System Programming Guide Part 2
3B
Intel® 64 and IA-32 Architectures Software Developer's Manual
VOLUME 3A: System Programming Guide Part 1
3A
Intel® 64 and IA-32 Architectures Software Developer's Manual
VOLUME 2B: Instruction Set Reference, N-Z
2B
Intel® 64 and IA-32 Architectures Software Developer's Manual
VOLUME 2A: Instruction Set Reference, A-M
2A
Intel® 64 and IA-32 Architectures Software Developer's Manual
VOLUME 1: Basic Architecture
1

x86 Assembly Syntax

- **Two "flavors":**
  - Intel syntax (what's in the processor manual)
  - AT&T syntax (what GNU as uses)
- **The difference between them:**
  - Intel: mov eax, DWORD PTR [ebx] -> DEST, SRC
  - AT&T: movl (%ebx), %eax -> SRC, DEST

We will work with **AT&T syntax**

Data Movement

- mov <reg>,<reg>
- mov <reg>,<mem>
- mov <mem>,<reg>
- mov <reg>,<const>
- mov <mem>,<const>

- What's missing?

Data Movement

- **Examples (AT&T syntax)**
  - Source, dest
  - `movl %eax, %edx => edx = eax`
  - `movl $0x123, %ebx => ebx = 0x123`
  - `movl 0x123, %ebx => ebx = *(int *)0x123`
  - `movl (%ebx), %edx => edx = *(int *)ebx`
  - `movl 4(%ebx), %edx =>`
    - `edx = *(int *)(ebx + 4)`

Push/Pop

- Recall: ESP is stack pointer
- On x86 the stack grows *downward* – putting something on the stack decrements the stack pointer
- Push subtracts 4 from ESP, then writes to the memory at address ESP

Arithmetic

- add, sub, inc, dec
- imul, idiv
- and, or, xor
- not
- neg
- shl, shr

Logical

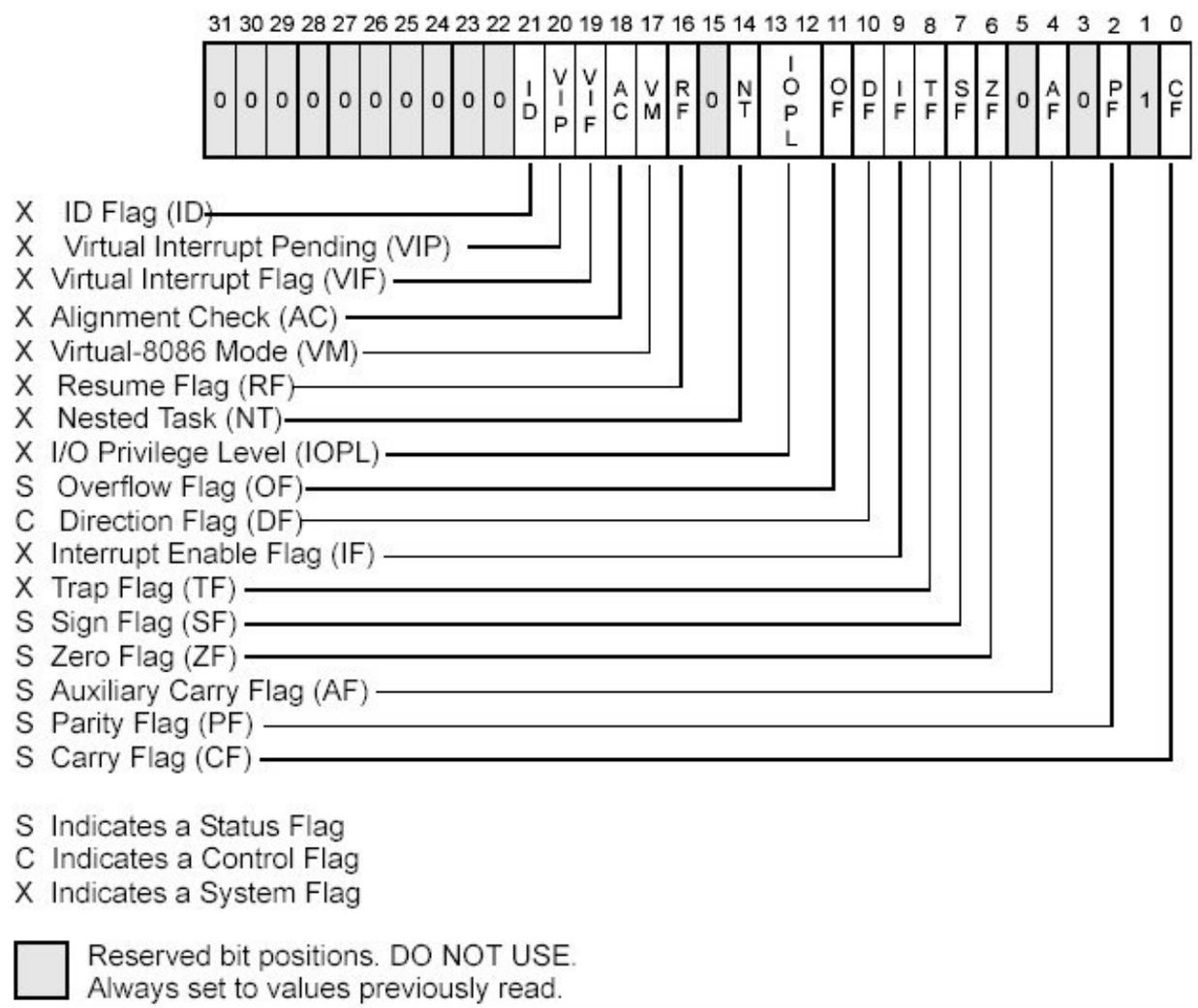
- test %x, %y – equivalent to BITWISE AND %x, %y but does not modify %y

- cmp %x, %y – equivalent to sub %x, %y but does not modify %y

- So if they don't modify the destination operand, what do they do?

EFLAGS

- **Arithmetic and logical instructions set bits in EFLAGS as a side effect**



EFLAGS Register

EFLAGS

- Conditional branches depend on the values in EFLAGS
- Also some specific instructions, e.g. cmov, will be executed only when flags are satisfied

Assembly Characteristics: Data Types

- **"Integer" data of 1, 2, 4, or 8 bytes**
  - Data values
  - Addresses (untyped pointers)

- **Floating point data of 4, 8, or 10 bytes**

- **Code: Byte sequences encoding series of instructions**

- **No aggregate types such as arrays or structures**
  - Just contiguously allocated bytes in memory

Assembly Characteristics: Operations

- **Perform arithmetic function on register or memory data**

- **Transfer data between memory and register**
  - Load data from memory into register
  - Store register data into memory

- **Transfer control**
  - Unconditional jumps to/from procedures
  - Conditional branches

Conditional Branches

Instruction	Description	signed-ness	Flags
JO	Jump if overflow		OF = 1
JNO	Jump if not overflow		OF = 0
JS	Jump if sign		SF = 1
JNS	Jump if not sign		SF = 0
JE **JZ**	Jump if equal Jump if zero		ZF = 1
JNE **JNZ**	Jump if not equal Jump if not zero		ZF = 0
JB **JNAE** **JC**	Jump if below Jump if not above or equal Jump if carry	unsigned	CF = 1
JNB **JAE** **JNC**	Jump if not below Jump if above or equal Jump if not carry	unsigned	CF = 0
JBE **JNA**	Jump if below or equal Jump if not above	unsigned	CF = 1 or ZF = 1
JA **JNBE**	Jump if above Jump if not below or equal	unsigned	CF = 0 and ZF = 0
JL **JNGE**	Jump if less Jump if not greater or equal	signed	SF <> OF

Unconditional Branch

- **jmp: unconditional jump**
  - jmp <const> – jump to constant address
  - jmp <mem> – dereference mem and jump
  - jmp <reg> - jump to address in register

String

- **None of them take an explicit operand; they work with implicity operands.**
  - **Source** is DS:SI
  - **Destination**: ES:DI
- **lodsb, stosb: loads or stores a byte from %esi or %edi and increments %esi/%edi**
- **movsb: move byte from (%esi) to (%edi) and increments both**
- **scasb: compares %(es:edi) with %al and sets flags, then increments %edi**
- **String operations can be used with the repCC prefix, which repeats the instruction while a condition CC is true**

System

- Many instructions control system behavior
- int – executes a *software interrupt* (traps into the kernel)
- iret – return from an interrupt
- sysenter/sysret – newer way to enter/leave kernel mode

Fill in the blanks

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax		
0x104		
$0x108		
(%rax)		
4(%rax)		

X86 ASSEMBLY LANGUAGE FUNDAMENTALS

Instructions

- **Assembled into machine code by assembler**
- **Executed at runtime by the CPU**
- **Member of the Intel IA-32 instruction set**
- **Four parts**
  - Label (optional)
  - Mnemonic (required)
  - Operand (usually required)
  - Comment (optional)

Label:	Mnemonic	Operand(s)	;Comment

Labels

- **Act as place markers**
  - marks the address (offset) of code and data
- **Easier to memorize and more flexible**

  `mov ax, [0020]` → `mov ax, val`
- **Follow identifier rules**

Labels

- **Data label**
  - must be unique
  - **example**: `myArray  BYTE  10`

- **Code label (ends with a colon)**
  - target of jump and loop instructions
  - **example**:

```
L1: mov ax, bx
    ...
    jmp L1
```

Reserved words and identifiers

- **Reserved words cannot be used as identifiers**
  - Instruction mnemonics, directives (TITLE, INCLUDE, NAME, LABEL…), type attributes, operators, predefined symbols
  - Full list of directives in the docs.

- **Identifiers**
  - 1-247 characters, including digits
  - case insensitive (by default)
  - first character must be a letter, `_`, `@`, or `$`
  - examples:

```
var1      Count        $first
_main          MAX     open_file
@@myfile xVal     _12345
```

Mnemonics and operands

- **Instruction mnemonics**
  - "reminder"
  - examples: **`MOV, ADD, SUB, MUL, INC, DEC`**
- **Operands**
  - constant (immediate value), **`96`**
  - constant expression, **`2+4`**
  - Register, **`eax`**
  - memory (data label), **`count`**
- **Number of operands: 0 to 3**
  - `stc ; set Carry flag`
  - `inc ax ; add 1 to ax`
  - `mov count, bx ; move BX to count`

Comments

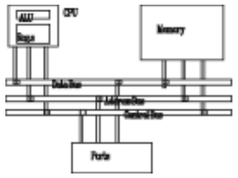
- **Comments are good!**
  - explain the program's purpose
  - tricky coding techniques
  - application-specific explanations

- **Single-line comments**
  - begin with semicolon (;)

- **block comments**
  - begin with **COMMENT** directive and a programmer-chosen character and end with the same programmer-chosen character

```
COMMENT !
   This is a comment
   and this line is also a comment
!
```

Example: adding/subtracting integers



directive marking a comment

```
TITLE Add and Subtract                    (AddSub.asm)

; This program adds and subtracts 32-bit integers.

INCLUDE nyu32.inc
.code
main PROC
    mov 10000h, eax          ; EAX = 10000h
    add 40000h, eax          ; EAX = 50000h
    sub 20000h, eax          ; EAX = 30000h
    call DumpRegs            ; display registers
    exit
main ENDP
END main
```

comment

copy definitions from nyu.inc

code segment. 3 segments: code, data, stack

beginning of a procedure

source

destination

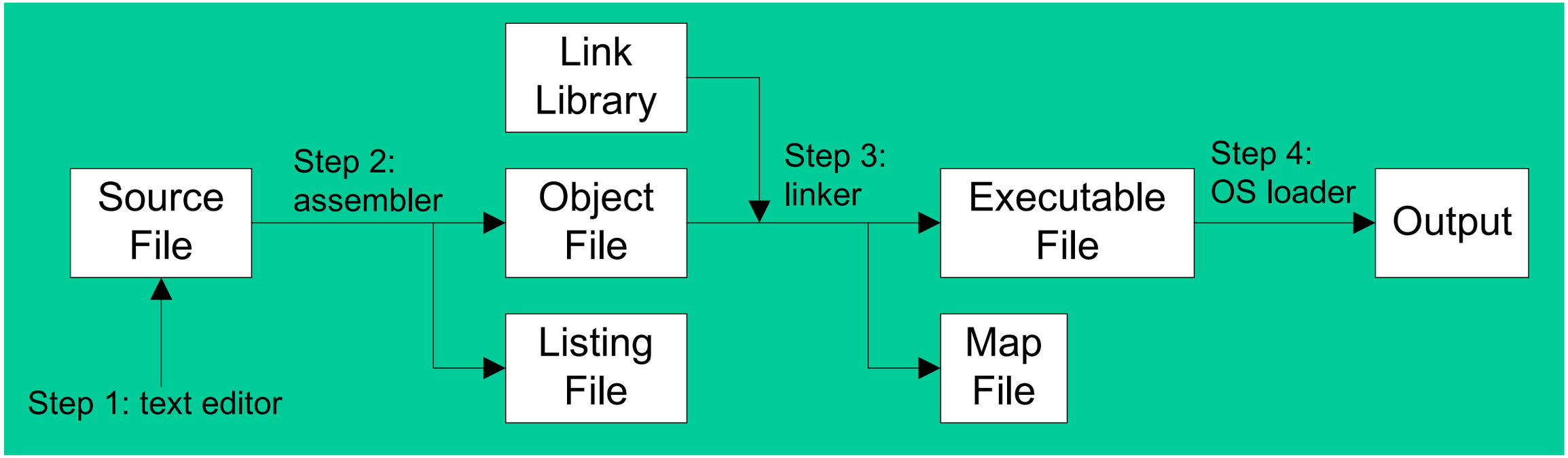
defined in nyu32.inc to end a program

marks the last line and
define the startup procedure

Example output

Program output, showing registers and flags:

```
EAX=00030000  EBX=7FFDF000  ECX=00000101  EDX=FFFFFFFF
ESI=00000000  EDI=00000000  EBP=0012FFF0  ESP=0012FFC4
  EIP=00401024  EFL=00000206  CF=0  SF=0  ZF=0  OF=0
```

Assemble-link execute cycle

- The following diagram describes the steps from creating a source program through executing the compiled program.
- If the source code is modified, Steps 2 through 4 must be repeated.